LLM Inference Gateway: Concurrency, Queue Management, and Fault Handling

PART- I

Alis

https://softmaxxer101.github.io/
https://x.com/import_alis

Date: 27th October 2026

Abstract- This blog documents the development of an inference gateway for LLM serving using FastAPI and vLLM. We are using Qwen2.5-0.5B-Instruct model. It explores concurrency limits, bounded queues, overload rejection, queue deadlines, upstream timeouts, request logging, and metrics. Through controlled load tests, I investigate how queue capacity affects latency and how the gateway handles timeouts and subsequent recovery.

Index Terms- LLM Inference, vLLM, FastAPI, Inference Gateway, Concurrency Control, Queue Management, Overload Handling, Timeouts, Load Testing, Observability.

I. THE GATEWAY EXPERIMENTS

THE GITHUB REPO LINK : HTTPS://GITHUB.COM/SOFTMAXXER101/INFERENCE_GATEWAY (ACCESS THE GATEWAY AND LOAD_TEST FOLDER FROM THIS REPO)

Start your gateway:
Open another terminal in your project directory.

#uvicorn gateway:app --port 9000

Experiment 1: Establish the baseline

#python load_test.py 1

```
{"gateway":"up","vllm":"up"}(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ python load_
python load_test.py 1
Sent 1 simultaneous requests -> status counts: {200: 1}
200s: median 0.64s, p95 0.64s, max 0.64s
```

Experiment 3: Test overload handling

#python load_test.py 10

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ python load_test.py 10
Sent 10 simultaneous requests -> status counts: {200: 5, 429: 5}
200s: median 1.37s, p95 2.04s, max 2.04s
429s: slowest rejection took 0.02s (should be near-instant for queue_full)
```

Visible mixture of HTTP 200 and 429 as given below. Our gateway had max active slot =1 and Queue waiting slot = 4, hence only 5 requests could be accepted and the rest 5 would be declined as given below.

```
INFO:     127.0.0.1:36232 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:36238 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:36240 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:36254 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:36266 - "POST /v1/chat/completions HTTP/1.1" 429 Too Many Requests
INFO:     127.0.0.1:36198 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:36204 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:36206 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:36216 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO:     127.0.0.1:36222 - "POST /v1/chat/completions HTTP/1.1" 200 OK
```

Next let's inspect our gateway's internal metrics

#curl -s http://localhost:9000/metrics

```
(venv) alis@LAPTOP-N3QNJPKH:~/project/production_vllm$ curl -s http://localhost:9000/metrics
{"received":21,"completed":11,"rejected_queue_full":10,"rejected_deadline":0,"upstream_timeout"
:0,"upstream_error":0,"waiting_now":0,"active_now":0,"max_active":1,"max_queue":4}(venv) alis@L
```

received:	21
completed:	11
rejected_queue_full:	10
rejected_deadline:	0
upstream_timeouts:	0
upstream_errors:	0
waiting_now:	0
active_now:	0
max_active:	1
max_queue:	4

Inspect your request logs

Your gateway writes one JSON record per request to >> results/requests.jsonl

tail -n 10 results/requests.jsonl

```
APTOP-N3QNJPKH:~/project/production_vllm$ tail -n 10 results/requests.jsonl
{"ts": 1793112168.6603177, "outcome": "queue_full", "status": 429, "queue_wait_s": 0.0012, "se
vice_s": 0.0, "waiting_at_exit": 4, "active_at_exit": 1}
{"ts": 1793112168.661689, "outcome": "queue_full", "status": 429, "queue_wait_s": 0.0, "servic
_s": 0.0, "waiting_at_exit": 4, "active_at_exit": 1}
{"ts": 1793112168.662051, "outcome": "queue_full", "status": 429, "queue_wait_s": 0.0, "servic
_s": 0.0, "waiting_at_exit": 4, "active_at_exit": 1}
{"ts": 1793112168.6624107, "outcome": "queue_full", "status": 429, "queue_wait_s": 0.0, "servi
e_s": 0.0, "waiting_at_exit": 4, "active_at_exit": 1}
{"ts": 1793112168.6627402, "outcome": "queue_full", "status": 429, "queue_wait_s": 0.0, "servi
e_s": 0.0, "waiting_at_exit": 4, "active_at_exit": 1}
{"ts": 1793112169.1671293, "outcome": "ok", "status": 200, "queue_wait_s": 0.0008, "service_s"
 0.5078, "waiting_at_exit": 4, "active_at_exit": 1}
{"ts": 1793112169.4449906, "outcome": "ok", "status": 200, "queue_wait_s": 0.5088, "service_s"
 0.2775, "waiting_at_exit": 3, "active_at_exit": 1}
{"ts": 1793112169.7586432, "outcome": "ok", "status": 200, "queue_wait_s": 0.7866, "service_s"
 0.3133, "waiting_at_exit": 2, "active_at_exit": 1}
{"ts": 1793112170.119253, "outcome": "ok", "status": 200, "queue_wait_s": 1.1002, "service_s":
0.3603, "waiting_at_exit": 1, "active_at_exit": 1}
{"ts": 1793112170.4369495, "outcome": "ok", "status": 200, "queue_wait_s": 1.4605, "service_s"
 0.3174, "waiting_at_exit": 0, "active_at_exit": 1}
```

The above command gives a summary of all the requests processed through the gateway. It writes down all the request records into a json file inside the result directory.

We should look at fields such as:

```
{
  "outcome": "ok",
  "status": 200,
  "queue_wait_s": 0.001,
```

```
  "service_s": 0.850,
  "waiting_at_exit": 0,
  "active_at_exit": 1
}
```

The above gives a brief log of each and every request in a similar format.

Box for reference for variables:

- queue_wait_s: time spent waiting for a gateway slot.
- service_s: time spent waiting for the upstream HTTP response.
- waiting_at_exit: requests still waiting when this event was logged.
- active_at_exit: requests being served when this event was logged.

Restart the gateway with explicit settings

```
# MAX_ACTIVE=1 MAX_QUEUE=4 QUEUE_DEADLINE=15 UPSTREAM_TIMEOUT=30
LOG_PATH=results/exp2_queue4.jsonl uvicorn gateway:app --port 9000
```

Run the following commands and log the outputs:

```
# python load_test.py 20
# curl -s http://localhost:9000/metrics
# cat results/exp2_queue4.jsonl
# tail -n 10 results/exp2_queue4.jsonl
```

1.Measured results:

Metric	**Result**
Concurrent requests sent	20
Successful responses	5
Queue-full rejections (429)	15
Queue-deadline rejections	0
Upstream timeouts	0
Upstream errors	0
Successful-request median latency	0.96 s
Successful-request P95 latency	1.56 s
Maximum successful-request latency	1.56 s
Configured active-request limit	1
Configured queue capacity	4

configured capacity was: MAX_ACTIVE + MAX_QUEUE = 1 + 4 = 5 requests
Observation 1- >The gateway: admitted five requests & rejected the other 15.
Observation 2-> Excess requests were rejected quickly
Observation 3->

Request	**-Queue wait**	**-Upstream service time**
1	0.0008 s	0.5270 s
2	0.5094 s	0.1794 s

Request	-Queue wait	-Upstream service time
3	0.6890 s	0.2205 s
4	0.9099 s	0.2765 s
5	1.1868 s	0.3269 s

The Queue_wait increases per request as the amount of time for waiting increases per request. The first upstream takes more time than second request's upstream service, this is usually because the first request takes more time to process.

2. Reduce queue capacity from 4 to 1

Restart the gateway by changing the variables:

MAX_ACTIVE=1 MAX_QUEUE=1 QUEUE_DEADLINE=15 UPSTREAM_TIMEOUT=30
LOG_PATH=results/exp2_queue1.jsonl uvicorn gateway:app --port 9000

Run the following commands and log the outputs:
python load_test.py 20
curl -s http://localhost:9000/metrics
cat results/exp2_queue1.jsonl

Results:

Metric	Queue = 4	Queue = 1	What changed?
Requests sent	20	20	Same workload
Successful responses	5	2	−3
Queue-full rejections	15	18	+3
Successful-request median latency	0.96 s	0.93 s	−0.03 s
Successful-request P95 latency	1.56 s	1.03 s	−0.53 s
Slowest rejected request	0.06 s	0.04 s	Both fast
Maximum active requests configured	1	1	Unchanged
Maximum queue capacity	4	1	Reduced

The above configuration confirms that MAX_ACTIVE=1 MAX_QUEUE=1, hence only 2 request will be accepted and rest 18 will be rejected. The above table shows comparision between Queue=4 vs Queue=1 experiments with changed variables. The current configuration only allows 2 requests, rest all are thrown to 429 error. Reducing the queue did not make inference faster instead it changed the number of requests admitted during the burst.

Ps- each configuration was tested only once and number of requests were also small, hence the difference between the p95 latency in queue=4 and queue=1 has a difference of -0.03s. This difference cannot be validated with less amount of requests and one run. To validate this, we need more robust percentile calculation and repeated runs.

3. Increase the queue to 9

Restart the gateway:

MAX_ACTIVE=1 MAX_QUEUE=9 QUEUE_DEADLINE=15 UPSTREAM_TIMEOUT=30
LOG_PATH=results/exp2_queue9.jsonl uvicorn gateway:app --port 9000

Run the following commands and log the outputs:
python load_test.py 20
curl -s http://localhost:9000/metrics
cat results/exp2_queue9.jsonl

Results: Final comparison: queue capacity versus latency

Metric	Queue = 1	Queue = 4	Queue = 9
Concurrent requests sent	20	20	20
Successful responses	2	5	10
Rejected requests	18	15	10
Success rate	10%	25%	50%
Median successful latency	0.93 s	0.96 s	2.48 s
Reported P95 latency	1.03 s	1.56 s	3.71 s
Maximum successful latency	1.03 s	1.56 s	3.71 s
Maximum queue capacity	1	4	9

Experiment 2 — Queue deadlines and latency protection

Restart the gateway:

MAX_ACTIVE=1 MAX_QUEUE=9 QUEUE_DEADLINE=1 UPSTREAM_TIMEOUT=30
LOG_PATH=results/exp3_deadline1.jsonl uvicorn gateway:app --port 9000

Run the following commands and log the outputs:
python load_test.py 20
curl -s http://localhost:9000/metrics
cat results/exp3_deadline1.jsonl

Results:

Outcome	Requests	Meaning
Successful (200)	3	Requests reached the inference server and succeeded
Queue full (429)	10	Rejected immediately because the gateway had reached its admission capacity
Queue deadline (429)	7	Rejected after waiting approximately 1 second for an active slot
Total	**20**	All requests accounted for

We have initialized two types of rejections in our gateway:
queue_full = once the queue is full, it can nomore accept further requests and further requests will be auto-rejected with 429 error.
Queue_deadline= any request waiting in the queue for more than the allotted queue_deadline time would be auto-rejected.

Experiment 3 — Handling inference-server timeouts

Setting	What it limits
QUEUE_DEADLINE	How long a request waits for an available gateway slot
UPSTREAM_TIMEOUT	How long the gateway waits for the inference server's response

Restart the gateway:

MAX_ACTIVE=1 MAX_QUEUE=4 QUEUE_DEADLINE=15 UPSTREAM_TIMEOUT=0.001
LOG_PATH=results/exp4_timeout.jsonl uvicorn gateway:app --port 9000

Run the following commands and log the outputs:
python load_test.py 20
curl -s http://localhost:9000/metrics
cat results/exp4_timeout.jsonl

Results:

Metric	Result
Requests sent	5
HTTP 504 responses	5
Upstream timeouts	5
Upstream errors	0
Queue-full rejections	0
Queue-deadline rejections	0
Active requests remaining	0
Waiting requests remaining	0
Maximum active requests	1

```
INFO:     127.0.0.1:60194 - "POST /v1/chat/completions HTTP/1.1" 504 Gateway Timeout
INFO:     127.0.0.1:60208 - "POST /v1/chat/completions HTTP/1.1" 504 Gateway Timeout
INFO:     127.0.0.1:60214 - "POST /v1/chat/completions HTTP/1.1" 504 Gateway Timeout
INFO:     127.0.0.1:60216 - "POST /v1/chat/completions HTTP/1.1" 504 Gateway Timeout
INFO:     127.0.0.1:60228 - "POST /v1/chat/completions HTTP/1.1" 504 Gateway Timeout
```

The above screenshot shows the 504 gateway timeout- which means the vllm did not respond on time/ took too long to respond.

The upstream_timeout which was previously set to 30sec was tuned to 0.001s and now all the 5 requests gave 504 gateway timeout.

Experiment 4 — Recovery after a timeout

Keep the same gateway running.

python load_test.py 1
It will probably again return a 504 gateway timeout.

Restart the gateway

MAX_ACTIVE=1 MAX_QUEUE=4 QUEUE_DEADLINE=15 UPSTREAM_TIMEOUT=30
LOG_PATH=results/exp5_recovery.jsonl uvicorn gateway:app --port 9000

Run the following commands and log the outputs:
python load_test.py 1
curl -s http://localhost:9000/metrics

Results:

Test	Configuration	Result
Timeout test	UPSTREAM_TIMEOUT=0.001	504
Recovery test	UPSTREAM_TIMEOUT=30	200
Recovery request latency	30-second timeout	0.91 seconds
Active requests after recovery	—	0
Waiting requests after recovery	—	0
Upstream timeouts during recovery test	—	0

The above table shows difference between upstream_timeout= 30s vs 0.001s. We have re-stored the original 30sec upstream_timeout in the last experiment and verified that all the setups are working.